Московский авиационный институт
(национальный исследовательский университет)

Факультет компьютерных наук и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Дискретный анализ»

Студент:	М. Г. Григорьев
Преподаватель:	С. А. Михайлова
Группа:	М8О-201БВ
Дата:	
Оценка:	
Подпись:	

Москва, 2026

Лабораторная работа №2

Задача: Требуется разработать программу-словарь на основе заданной структуры данных.

Вариант структуры данных: Красно-чёрное дерево.

Вариант ключа: Регистронезависимая последовательность букв английского алфавита длиной не более 256 символов.

Вариант значения: `Целое беззнаковое число от 0 до` $2^{64} - 1$.

Словарь должен поддерживать операции вставки, удаления, поиска, сохранения в бинарный файл и загрузки из бинарного файла. При загрузке корректного файла новый словарь должен заменять текущий; при ошибке чтения или несовпадении формата рабочий словарь должен оставаться без изменений. Пустой или отсутствующий файл при загрузке считается корректным и соответствует пустому словарю.

1 Описание

В работе реализован словарь на основе собственного красно-чёрного дерева. Красно-чёрное дерево представляет собой бинарное дерево поиска, в котором каждая вершина имеет цвет, а структура поддерживает набор инвариантов, обеспечивающих логарифмическую высоту дерева и, как следствие, логарифмическое время основных операций [1].

Каждая вершина дерева хранит:

- строковый ключ;
- связанное с ним значение типа `uint64_t`;
- цвет вершины;
- указатели на родителя, левого и правого потомка.

Для упрощения реализации пустых детей и пустого дерева используется специальная чёрная служебная вершина `nil_`. Это позволяет не работать с `nullptr` в коде балансировки и записывать алгоритмы вставки и удаления в форме, близкой к классическому описанию [1, 2].

Перед каждой операцией ключ переводится в нижний регистр, поэтому строки `word`, `Word` и `WORD` считаются одним и тем же ключом.

1 Поиск

Поиск выполняется как в обычном бинарном дереве поиска: начиная с корня, ключ сравнивается с текущей вершиной, после чего выполняется переход в левое или правое поддерево. Если достигнут `nil_`, слово в словаре отсутствует. Время работы поиска равно $O(\log n)$.

2 Вставка

Вставка разбита на две части. Сначала новый элемент вставляется в дерево как в обычное бинарное дерево поиска. Если такой ключ уже существует, операция завершается без изменения словаря. Новый узел создаётся красным, после чего вызывается процедура `InsertFixup`, восстанавливающая инварианты красно-чёрного дерева.

В процедуре балансировки рассматриваются стандартные случаи:

- красный дядя — выполняется перекраска родителя, дяди и деда;

- чёрный дядя и «ломаная» конфигурация — сначала выполняется поворот вокруг родителя;
- чёрный дядя и «прямая» конфигурация — выполняется поворот вокруг деда и перекраска.

В конце корень принудительно окрашивается в чёрный цвет.

3 Удаление

Удаление также состоит из двух этапов. Сначала вершина удаляется по правилам бинарного дерева поиска:

- если у неё не более одного непустого потомка, выполняется простая замена поддерева;
- если есть оба потомка, на место удаляемой вершины переносится её inorder-последователь — минимальная вершина в правом поддереве.

Если фактически удалённая вершина была чёрной, вызывается `DeleteFixup`. В этой процедуре рассматриваются случаи по цвету брата и его потомков. Используются перекраски и повороты, пока не будет восстановлена одинаковая чёрная высота на всех путях от текущей вершины к листьям.

4 Сохранение и загрузка

Для сохранения словаря реализован компактный бинарный формат. В файл записываются:

1. сигнатура формата `RBTD`;
2. версия формата;
3. число элементов в словаре;
4. для каждого элемента: длина ключа, байты ключа и соответствующее значение.

Числовые поля записываются явно в little-endian виде, поэтому формат не зависит от внутреннего представления чисел в памяти.

Загрузка выполняется во временный объект словаря. Это сделано специально: если файл повреждён, имеет неверный формат или при чтении произошла системная ошибка, текущий словарь не изменяется. Только после полного успешного чтения вызывается `Swap`, и временный словарь подменяет рабочий.

При загрузке выполняются дополнительные проверки:

- совпадение сигнатуры и версии;
- корректная длина ключа;
- допустимость символов ключа;
- отсутствие лишних байтов в конце файла;
- отсутствие повторяющихся ключей.

Отсутствующий или пустой файл интерпретируется как корректный пустой словарь, что соответствует условию задачи.

5 Структура программы

Программа целиком размещена в одном файле `main.cpp`. Центральной частью является класс `RedBlackTreeDictionary`, который инкапсулирует:

- внутреннее представление узлов;
- операции поиска, вставки и удаления;
- повороты и процедуры балансировки;
- функции двоичной сериализации и десериализации.

Функция `main` отвечает только за консольный интерфейс: считывает команды, нормализует ключи, вызывает соответствующие методы словаря и печатает результат в формате, заданном условием.
Асимптотика реализованного словаря:

- поиск, вставка и удаление — $O(\log n)$;
- сохранение и загрузка — $O(n)$;
- дополнительная память структуры — $O(n)$.

2 Исходный код

Ниже приведены ключевые фрагменты реализации, отражающие устройство словаря, модификацию дерева и двоичную работу с файлом.

Листинг 1: Структура словаря и служебные объекты

```
class RedBlackTreeDictionary {
private:
    enum Color {
        RED,
        BLACK
    };

    struct Node {
        std::string key;
        uint64_t value;
        Color color;
        Node* parent;
        Node* left;
        Node* right;

        Node()
            : value(0), color(BLACK), parent(nullptr), left(nullptr), right(nullptr) {
        }

        Node(std::string key_value, uint64_t stored_value, Color node_color, Node* nil)
            : key(std::move(key_value)),
              value(stored_value),
              color(node_color),
              parent(nil),
              left(nil),
              right(nil) {
        }
    };

    class FileHandle {
    public:
        explicit FileHandle(std::FILE* file = nullptr) : file_(file) {
        }

        ~FileHandle() {
            if (file_ != nullptr)
                std::fclose(file_);
        }

        std::FILE* get() const {
            return file_;
        }

        int Close() {
            if (file_ == nullptr)
                return 0;
            std::FILE* raw = file_;
```

```
            file_ = nullptr;
            return std::fclose(raw);
        }

    private:
        std::FILE* file_;
    };

    enum class ReadStatus {
        OK,
        END_OF_FILE,
        SYSTEM_ERROR
    };

    static constexpr unsigned char kMagic[4] = {'R', 'B', 'T', 'D'};
    static constexpr unsigned char kVersion = 1;

public:
    RedBlackTreeDictionary()
        : root_(nullptr), nil_(new Node()), size_(0) {
        nil_->color = BLACK;
        nil_->parent = nil_;
        nil_->left = nil_;
        nil_->right = nil_;
        root_ = nil_;
    }

    RedBlackTreeDictionary(const RedBlackTreeDictionary&) = delete;
    RedBlackTreeDictionary& operator=(const RedBlackTreeDictionary&) = delete;

    ~RedBlackTreeDictionary() {
        Clear();
        delete nil_;
    }

    void Swap(RedBlackTreeDictionary& other) noexcept {
        std::swap(root_, other.root_);
        std::swap(nil_, other.nil_);
        std::swap(size_, other.size_);
    }
```

Листинг 2: Вставка элемента в дерево

```
    bool Insert(std::string key, uint64_t value) {
        Node* parent = nil_;
        Node* current = root_;

        while (current != nil_) {
            parent = current;
```

```
        if (key == current->key)
            return false;
        current = (key < current->key) ? current->left : current->right;
    }

    Node* inserted = new Node(std::move(key), value, RED, nil_);
    inserted->parent = parent;

    if (parent == nil_)
        root_ = inserted;
    else if (inserted->key < parent->key)
        parent->left = inserted;
    else
        parent->right = inserted;

    InsertFixup(inserted);
    ++size_;
    return true;
}
```

Листинг 3: Удаление элемента из дерева

```
bool Erase(const std::string& key) {
    Node* target = FindNode(key);
    if (target == nil_)
        return false;

    Node* replacement_parent_candidate = target;
    Color original_color = replacement_parent_candidate->color;
    Node* fixup_node = nil_;

    if (target->left == nil_) {
        fixup_node = target->right;
        Transplant(target, target->right);
    } else if (target->right == nil_) {
        fixup_node = target->left;
        Transplant(target, target->left);
    } else {
        replacement_parent_candidate = Minimum(target->right);
        original_color = replacement_parent_candidate->color;
        fixup_node = replacement_parent_candidate->right;

        if (replacement_parent_candidate->parent == target) {
            fixup_node->parent = replacement_parent_candidate;
        } else {
            Transplant(replacement_parent_candidate, replacement_parent_candidate->
                right);
            replacement_parent_candidate->right = target->right;
            replacement_parent_candidate->right->parent =
```

```
                replacement_parent_candidate;
        }

        Transplant(target, replacement_parent_candidate);
        replacement_parent_candidate->left = target->left;
        replacement_parent_candidate->left->parent = replacement_parent_candidate;
        replacement_parent_candidate->color = target->color;
    }

    delete target;
    --size_;

    if (original_color == BLACK)
        DeleteFixup(fixup_node);

    return true;
```

Листинг 4: Публичные методы сохранения и загрузки

```
bool Save(const std::string& path, std::string& error) const {
    error.clear();

    FileHandle file(std::fopen(path.c_str(), "wb"));
    if (file.get() == nullptr) {
        error = MakeSystemErrorMessage(errno);
        return false;
    }

    if (!WriteBytes(file.get(), kMagic, sizeof(kMagic), error) ||
        !WriteByte(file.get(), kVersion, error) ||
        !WriteUint64(file.get(), size_, error) ||
        !WriteInOrder(file.get(), root_, error))
        return false;

    if (file.Close() != 0) {
        error = MakeSystemErrorMessage(errno);
        return false;
    }

    return true;
}

bool Load(const std::string& path, std::string& error) {
    error.clear();

    FileHandle file(std::fopen(path.c_str(), "rb"));
    if (file.get() == nullptr) {
        const int open_error = errno;
        if (open_error == ENOENT) {
```

```
            RedBlackTreeDictionary empty_dictionary;
            Swap(empty_dictionary);
            return true;
        }
        error = MakeSystemErrorMessage(open_error);
        return false;
    }

    RedBlackTreeDictionary loaded_dictionary;
    if (!ReadFromFile(file.get(), loaded_dictionary, error))
        return false;

    if (file.Close() != 0) {
        error = MakeSystemErrorMessage(errno);
        return false;
    }

    Swap(loaded_dictionary);
    return true;
}
```

Листинг 5: Двоичная сериализация и проверка формата файла

```
static ReadStatus ReadBytes(std::FILE* file, void* buffer, std::size_t size, std::
    string& error) {
    if (size == 0)
        return ReadStatus::OK;

    const std::size_t bytes_read = std::fread(buffer, 1, size, file);
    if (bytes_read == size)
        return ReadStatus::OK;

    if (std::ferror(file) != 0) {
        error = MakeSystemErrorMessage(errno);
        return ReadStatus::SYSTEM_ERROR;
    }

    return ReadStatus::END_OF_FILE;
}

static bool WriteBytes(std::FILE* file, const void* buffer, std::size_t size, std::
    string& error) {
    if (size == 0)
        return true;

    if (std::fwrite(buffer, 1, size, file) != size) {
        error = MakeSystemErrorMessage(errno);
        return false;
    }
```

```

        return true;
    }

    static bool WriteByte(std::FILE* file, unsigned char value, std::string& error) {
        return WriteBytes(file, &value, sizeof(value), error);
    }

    static bool WriteUint16(std::FILE* file, uint16_t value, std::string& error) {
        unsigned char buffer[2] = {
            static_cast<unsigned char>(value & 0xFFu),
            static_cast<unsigned char>((value >> 8) & 0xFFu)
        };
        return WriteBytes(file, buffer, sizeof(buffer), error);
    }

    static bool WriteUint64(std::FILE* file, uint64_t value, std::string& error) {
        unsigned char buffer[8];
        for (int i = 0; i < 8; ++i)
            buffer[i] = static_cast<unsigned char>((value >> (i * 8)) & 0xFFu);
        return WriteBytes(file, buffer, sizeof(buffer), error);
    }

    static ReadStatus ReadUint16(std::FILE* file, uint16_t& value, std::string& error)
        {
        unsigned char buffer[2];
        const ReadStatus status = ReadBytes(file, buffer, sizeof(buffer), error);
        if (status != ReadStatus::OK)
            return status;
        value = static_cast<uint16_t>(buffer[0]) |
                (static_cast<uint16_t>(buffer[1]) << 8);
        return ReadStatus::OK;
    }

    static ReadStatus ReadUint64(std::FILE* file, uint64_t& value, std::string& error)
        {
        unsigned char buffer[8];
        const ReadStatus status = ReadBytes(file, buffer, sizeof(buffer), error);
        if (status != ReadStatus::OK)
            return status;

        value = 0;
        for (int i = 0; i < 8; ++i)
            value |= static_cast<uint64_t>(buffer[i]) << (i * 8);
        return ReadStatus::OK;
    }

    static ReadStatus ReadExpected(std::FILE* file, void* buffer, std::size_t size, std
        ::string& error) {
```

```
    const ReadStatus status = ReadBytes(file, buffer, size, error);
    if (status == ReadStatus::END_OF_FILE)
        error = MakeInvalidFormatMessage();
    return status;
}

bool ReadFromFile(std::FILE* file, RedBlackTreeDictionary& loaded_dictionary, std::
    string& error) {
    int first_byte = std::fgetc(file);
    if (first_byte == EOF) {
        if (std::ferror(file) != 0) {
            error = MakeSystemErrorMessage(errno);
            return false;
        }

        RedBlackTreeDictionary empty_dictionary;
        loaded_dictionary.Swap(empty_dictionary);
        return true;
    }

    unsigned char magic[4];
    magic[0] = static_cast<unsigned char>(first_byte);
    ReadStatus status = ReadExpected(file, magic + 1, sizeof(magic) - 1, error);
    if (status != ReadStatus::OK)
        return false;

    for (std::size_t i = 0; i < sizeof(magic); ++i) {
        if (magic[i] != kMagic[i]) {
            error = MakeInvalidFormatMessage();
            return false;
        }
    }

    unsigned char version = 0;
    status = ReadExpected(file, &version, sizeof(version), error);
    if (status != ReadStatus::OK)
        return false;
    if (version != kVersion) {
        error = MakeInvalidFormatMessage();
        return false;
    }

    uint64_t count = 0;
    status = ReadUint64(file, count, error);
    if (status == ReadStatus::END_OF_FILE) {
        error = MakeInvalidFormatMessage();
        return false;
    }
    if (status == ReadStatus::SYSTEM_ERROR)
```

```
            return false;

        for (uint64_t i = 0; i < count; ++i) {
            uint16_t length = 0;
            status = ReadUint16(file, length, error);
            if (status == ReadStatus::END_OF_FILE) {
                error = MakeInvalidFormatMessage();
                return false;
            }
            if (status == ReadStatus::SYSTEM_ERROR)
                return false;
            if (length == 0 || length > 256) {
                error = MakeInvalidFormatMessage();
                return false;
            }

            std::string key(length, '\0');
            status = ReadExpected(file, key.data(), key.size(), error);
            if (status != ReadStatus::OK)
                return false;

            for (char& symbol : key) {
                if (!IsEnglishLetter(symbol)) {
                    error = MakeInvalidFormatMessage();
                    return false;
                }
                symbol = ToLowerAscii(symbol);
            }

            uint64_t value = 0;
            status = ReadUint64(file, value, error);
            if (status == ReadStatus::END_OF_FILE) {
                error = MakeInvalidFormatMessage();
                return false;
            }
            if (status == ReadStatus::SYSTEM_ERROR)
                return false;

            if (!loaded_dictionary.Insert(std::move(key), value)) {
                error = MakeInvalidFormatMessage();
                return false;
            }
        }

        const int extra_byte = std::fgetc(file);
        if (extra_byte != EOF) {
            error = MakeInvalidFormatMessage();
            return false;
        }
```

```
        if (std::ferror(file) != 0) {
            error = MakeSystemErrorMessage(errno);
            return false;
        }

        return true;
    }
```

3 Консоль

```
$ g++ -std=c++17 -O2 main.cpp -o lab2.out
$ ./lab2.out
+ a 1
OK
+ A 2
Exist
+ zebra 17
OK
zEbRa
OK: 17
! Save dict.bin
OK
- zebra
OK
! Load dict.bin
OK
zebra
OK: 17
```

4 Тест производительности

Для оценки производительности реализован отдельный бенчмарк, сравнивающий словарь на собственном красно-чёрном дереве со стандартным `std::map`. В измерение входят только операции над структурами данных `Insert`, `Find` и `Erase`; загрузка и сохранение в файл не тестируются, так как они зависят уже не только от структуры словаря, но и от работы файловой системы.

Тестовые данные генерируются скриптом `generator.py`. Для каждого запуска сначала выполняется `prefill = max(5000, n / 10)` стартовых вставок без измерения времени, после чего измеряется серия из n операций. Для каждого размера и сценария выполнено 5 прогонов, в таблицах приведены медианные значения времени в микросекундах. В последнем столбце указано отношение `std::map / RB`; значение больше 1 означает, что собственная реализация быстрее.

Рассматривались три сценария нагрузки:

- `mixed`: 50% поисков, 25% вставок, 25% удалений;
- `read-heavy`: 80% поисков, 10% вставок, 10% удалений;
- `update-heavy`: 40% вставок, 40% удалений, 20% поисков.

Команда запуска полной серии измерений:

```
$ make benchmark
$ bash run_benchmarks.sh benchmark_results.csv
```

Смешанный сценарий:

n	RB-tree, us	std::map, us	std::map / RB
50 000	20 026	20 811	1.039×
100 000	88 916	153 207	1.723×
200 000	185 440	211 016	1.138×
500 000	292 434	332 745	1.138×
1 000 000	1 327 619	1 292 859	0.974×

В смешанном сценарии собственная реализация оказалась быстрее в 4 случаях из 5. Наиболее заметный выигрыш получился при $n = 100\,000$, где медианное время оказалось лучше примерно в 1.72×. На максимальном размере входа результаты почти сравнялись, и небольшое преимущество осталось уже у `std::map`.

Сценарий с преобладанием поиска:

n	RB-tree, us	std::map, us	std::map / RB
50 000	20 361	19 646	0.965×
100 000	46 539	48 844	1.050×
200 000	86 936	91 528	1.053×
500 000	437 522	487 610	1.114×
1 000 000	1 083 802	1 110 723	1.025×

При преобладании операций поиска обе реализации показывают очень близкие результаты. Начиная с 10^5 операций собственное дерево немного опережает `std::map`, но преимущество здесь уже не столь велико: в основном речь идёт о нескольких процентах.

Сценарий с преобладанием модификаций:

n	RB-tree, us	std::map, us	std::map / RB
50 000	22 733	24 350	1.071×
100 000	49 666	49 653	1.000×
200 000	187 465	145 675	0.777×
500 000	417 693	387 555	0.928×
1 000 000	2 104 113	1 952 622	0.928×

Наиболее трудным для собственной реализации оказался сценарий, в котором доминируют вставки и удаления. На малых входах различие почти отсутствует, но начиная с $n = 200\,000$ стандартная библиотечная реализация работает быстрее. Это показывает, что одинаковая асимптотика $O(\log n)$ ещё не гарантирует одинаковые константы: при большом числе модифицирующих операций качество конкретной реализации становится особенно заметным.

В целом, результаты можно считать хорошими: собственное красно-чёрное дерево оказалось конкурентоспособным по сравнению с `std::map` и в двух сценариях из трёх чаще показывало равное или лучшее время. Одновременно бенчмарк показал и ограничение реализованного решения: под интенсивной нагрузкой на вставку и удаление стандартная библиотека обычно выигрывает за счёт более отлаженной и оптимизированной реализации.

5 Выводы

В этой лабораторной работе я реализовал словарь на собственном красно-чёрном дереве и довёл его до полного набора операций из условия: поиск, вставка, удаление, сохранение и загрузка. На практике самым важным оказалось не знание определения структуры, а аккуратная реализация изменяющих операций: нужно корректно менять связи между вершинами, обрабатывать крайние случаи и только после этого восстанавливать балансировку.

Самой содержательной частью работы оказалось удаление. Именно на этой операции стало понятно, насколько важны служебная вершина `nil_`, корректные указатели на родителей и чёткое разделение между обычной BST-частью алгоритма и последующей балансировкой через `DeleteFixup`. Вставка устроена проще, но тоже требует аккуратности, чтобы после поворотов и перекрасок дерево оставалось в корректном состоянии.

Дополнительную практическую ценность дал бенчмарк. Он показал, что собственная реализация не только работает корректно, но и остаётся конкурентоспособной по сравнению со `std::map`: в смешанном и read-heavy сценариях результаты чаще были не хуже, а в сценарии с преобладанием изменений стандартная библиотека ожидаемо оказалась сильнее. Для меня это полезный итог, потому что он связывает теорию с практикой: важно не только реализовать структуру данных, но и понимать, как она ведёт себя на разных типах нагрузки.

Для меня основной результат этой лабораторной в том, что я не просто разобрал красно-чёрное дерево по теории, а реализовал его как рабочий класс на C++ и отработал детали, от которых реально зависит надёжность программы. В итоге получилась программа, которая корректно выполняет все операции словаря из задания и даёт понятное представление о том, где собственная реализация уже сопоставима со стандартной, а где ещё уступает ей по качеству исполнения.

Список литературы

[1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 3-е издание.* — М.: Вильямс.

[2] Дональд Э. Кнут. *Искусство программирования. Том 3. Сортировка и поиск.* — М.: Вильямс.